

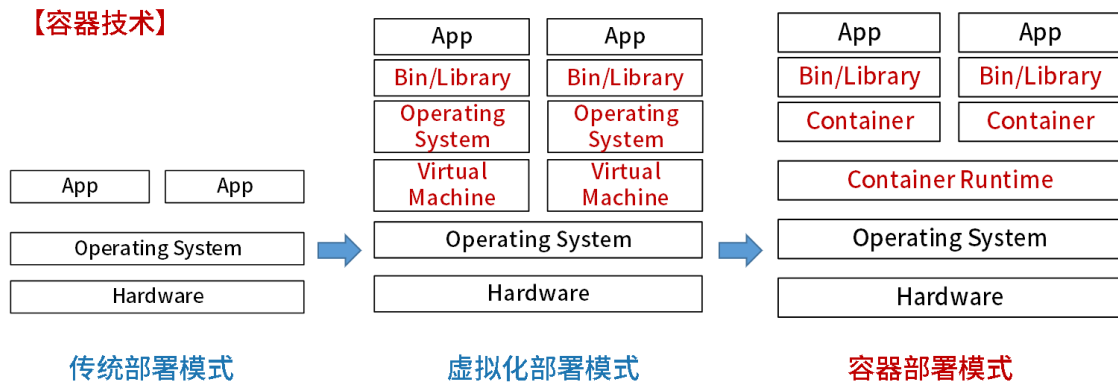
2026 年 4 月 8 日架构师网络课程纪要

一、本次课程总结

【课堂主题】

《案例特训（架构篇）》

【重点内容】



Docker容器基于操作系统虚拟化技术，共享操作系统内核、轻量、没有资源损耗、秒级启动，极大提升了系统的应用部署密度和弹性。



【虚拟机技术与容器技术的对比】

对比项	虚拟机技术	容器技术
镜像大小	包含GuestOS，G量级以上	仅包含运行的Bin/Lib，M量级
资源要求	CPU与内存按核、按G分配	CPU与内存按单核、低于G量级分配
启动时间	分钟级	毫秒级
可持续性	跨物理机迁移	跨操作系统平台迁移
弹性伸缩	VM伸缩，CPU/内存手动伸缩	实例自动伸缩、CPU内存自动在线伸缩
隔离策略	操作系统、系统级别	Cgroups，进程级别



边缘计算

【边缘计算】是指在靠近物或数据源头的一侧，采用网络、计算、存储、应用核心能力为一体的开放平台，就近提供最近端服务。

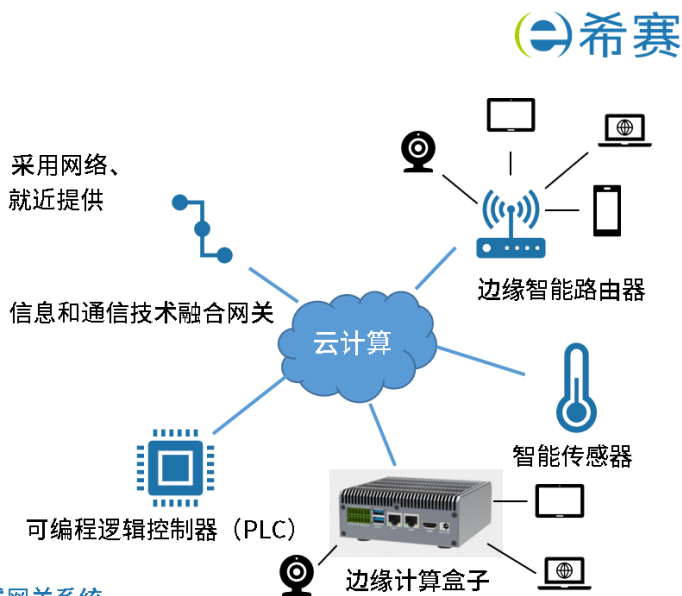
【边缘计算的本质】计算处理职能的本地化。

【边缘计算类型】

【云边缘】云服务在边缘侧的延伸

【边缘云】在边缘侧构建中小规模云服务能力

【边缘网关】以云化技术与能力重构原有嵌入式网关系统



边缘计算

【边云协同的分类】

【资源协同】边缘节点有基础设施资源的调度管理能力，可与云端协同。

【数据协同】边缘节点采集数据并初步分析，再发给云端做进一步处理。

【智能协同】分布式智能，云端做集中式模型训练，再将模型下发到边缘节点。

【应用管理协同】边缘节点提供应用部署与运行环境，云端主要提供应用开发、测试环境。

【业务管理协同】边缘节点提供模块化、微服务化的应用/数字孪生/网络等应用实例；云端主要提供按照客户需求实现应用/数字孪生/网络等的业务编排能力。

【服务协同】边缘节点按照云端策略实现部分ECSaaS服务，通过ECSaaS与云端SaaS的协同实现面向客户的按需SaaS服务；云端主要提供SaaS服务在云端和边缘节点的服务分布策略，以及云端承担的SaaS服务能力。



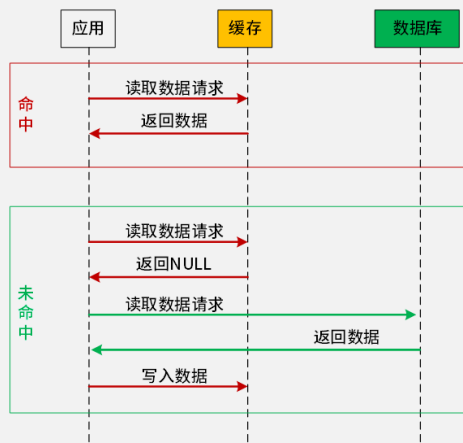
大型网站系统架构演化

维度	涉及技术内容
从架构来看	MVC, MVP, MVVM, REST, Webservice, 微服务
从并发分流来看	集群（负载均衡）、CDN
从缓存来看	MemCache, Redis, Squid
从数据来看	主从库（主从复制），内存数据库，反规范化技术，NoSQL，分区（分表）技术，视图与物化视图
从持久化来看	Hibernate, Mybatis
从分布存储来看	Hadoop, FastDFS, 区块链
从数据编码来看	XML, JSON
从Web应用服务器来看	Apache, WebSphere, WebLogic, Tomcat, JBOSS, IIS
从安全性来看	SQL注入攻击
其他	静态化，有状态与无状态，响应式Web设计，中台



缓存与数据库的数据一致性问题

Cache-Aside pattern 【旁路缓存模式】



【数据读取操作步骤】

- 1、根据key从缓存读取
- 2、若缓存中没有，则根据key在数据库中查找
- 3、读取到“值”之后，更新缓存

【代码示例】

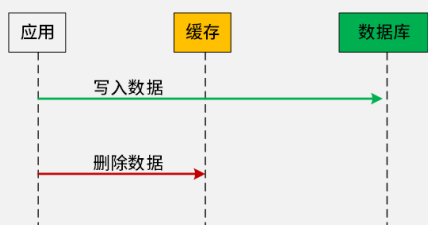
```
data = queryDataRedis(key);
if (data == null) {
    data = queryDataMySQL(key); // 缓存查询不到，从MySQL做查询
    if (data != null) {
        updateRedis(key, data); // 查询完数据后更新到Redis
    }
}
```



缓存与数据库的数据一致性问题



Cache-Aside pattern 【旁路缓存模式】



【数据写入操作步骤】

- 1、根据key值写数据库
- 2、根据key删除缓存

【代码示例】

```
updateMySQL();//更新MySQL数据库
deleteRedis(key, data);//删除缓存
```

【优点】按需加载数据到缓存，减少数据库的直接访问次数，提高性能。

【缺点】存在一致性问题。



缓存与数据库的数据一致性问题



【三种经典缓存使用模式多维度排名】

缓存模式	性能	数据一致性	数据可靠性	实时性
Cache-Aside	🥉 第2名	🥉 第2名	🥉 第2名	🥉 第2名
Read/Write-Through	🥉 第3名	🥇 第1名	🥇 第1名	🥇 第1名
Write-Behind	🥇 第1名	🥉 第3名	🥉 第3名	🥉 第3名



Redis数据分片方案

分片方案	分片方式	说明
范围分片	按数据范围值来做分片	例：按用户编号分片，0-999999映射到实例A；1000000-1999999映射到实例B。
哈希分片	通过对key进行hash运算分片	可以把数据分配到不同实例，这类似于取余操作，余数相同的，放在一个实例上。
一致性哈希分片	哈希分片的改进	利于扩展结点，可以有效解决重新分配节点带来的无法命中问题。



缓存技术对比【MemCache与Redis】

Redis 与 Memcache 能力比较

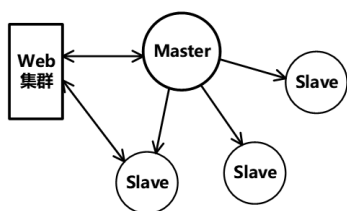
工作	MemCache	Redis
数据类型	简单key/value结构	丰富的数据结构
持久性	不支持	支持
分布式存储	客户端哈希分片/一致性哈希	多种方式，主从、Sentinel、Cluster等
多线程支持	支持	支持（Redis5.0及以前版本不支持）
内存管理	私有内存池/内存池	无
事务支持	不支持	有限支持
数据容灾	不支持，不能做数据恢复	支持，可以在灾难发生时，恢复数据



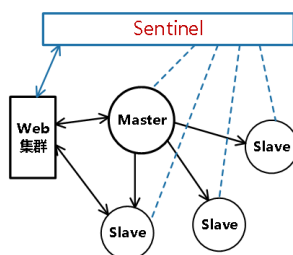
Redis分布式存储方案



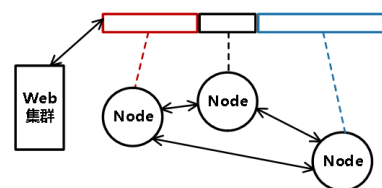
分布式存储方案	核心特点
主从（Master/Slave）模式	一主多从，故障时手动切换
哨兵（Sentinel）模式	有哨兵的一主多从，主节点故障自动选择新的主节点
集群（Cluster）模式	分节点对等集群，分slots，不同slots的信息存储到不同节点。



主从（Master/Slave）模式



哨兵（Sentinel）模式



集群（Cluster）模式



Redis数据分片方案



分片方案	分片方式	说明
范围分片	按数据范围值来做分片	例：按用户编号分片，0-999999映射到实例A；1000000-1999999映射到实例B。
哈希分片	通过对key进行hash运算分片	可以把数据分配到不同实例，这类似于取余操作，余数相同的，放在一个实例上。
一致性哈希分片	哈希分片的改进	利于扩展节点，可以有效解决重新分配节点带来的无法命中问题。

【视频回看地址】

希赛网目前提供了两种课程回看的方式：

方式一：课程回放【课程结束后 20 分钟内可立即回看】

APP 观看路径：我的->网络课堂

PC 观看路径：学习中心->网络课堂

回看注意事项：

(1) 首先登录希赛网(<https://www.xisaiwang.com/>)，然后使用以上地址回看。

(2) 回看时如果需要拖放观看,有两种方法进行视频内容定位:方法一是将鼠标移至视频头像位置进行拖放;方法二是直接点击播放界面下方的微缩图,可直接定位到这一页 PPT 的讲解。

方式二:老师端录屏回看【课程结束后 1-2 个工作日发布】

APP 观看路径:我的->视频课程

PC 观看路径:学习中心->视频课程

二、下次课程安排

主题:《案例特训(架构篇)》+ 人工智能

时间:2026 年 4 月 11 日【周六】9:00-12:00 14:00-17:00

预习要求:

回顾软件架构设计章节知识内容。

视频地址为:

<https://wangxiao.xisaiwang.com/nstu/sub/course/courseDetail/courseDetail?courseId=13147>

三、课后作业

某软件公司为某品牌手机厂商开发一套手机应用程序集成开发环境,以提高开发手机应用程序的质量和效率。在项目之初,公司的系统分析师对该集成开发环境的需求进行了调研和分析,具体描述如下:

- a. 需要同时支持该厂商自行定义的应用编程语言的编辑、界面可视化设计、编译、调试等模块,这些模块产生的模型或数据格式差异较大,集成环境应提供数据集成能力。集成开发环境还要支持以适配方式集成公司现有的应用模拟器工具。
- b. 经过调研,手机应用开发人员更倾向于使用 Windows 系统,因此集成开发环境的界面需要与 Windows 平台上的主流开发工具的界面风格保持一致。
- c. 支持相关开发数据在云端存储,需要在云端存储数据的机密性和完整性。
- d. 支持用户通过配置界面依据自己的喜好修改界面风格,包括颜色、布局、代码高亮方式等,配置完成后无需重启环境。
- e. 支持不同模型的自动转换。在初始需求中定义的机器性能条件下,对于一个包含 50 个对象的设计模型,将其转换为相应代码框架时所消耗时间不超过 5 秒。
- f. 能够连续运行的时间不小于 240 小时,意外退出后能够在 10 秒之内自动重启。
- g. 集成开发环境具有模块化结构,支持以模块为单位进行调试、测试与发布。
- h. 支持应用开发过程中的代码调试功能:开发人员可以设置断点,启动调试,编辑器可以自动卷屏并命中断点,能通过变量监视器查看当前变量取值。

在对需求进行分析后,公司的架构师小张查阅了相关的资料,认为该集成开发环境应该采用管道一过滤器(Pipe-Filter)的架构风格,公司的资深架构师王工在仔细分析后,认为应该采用数据仓储(Data

Repository) 的架构风格。公司经过评审，最终采用了王工的方案。

【问题 1】 (10 分)

识别软件架构质量属性是进行架构设计的重要步骤。请分析题干中的需求描述，填写表 1-1 中 (1) ~ (5) 处的空白。

质量属性名称	需求描述编号
可用性	(1)
(2)	e
可修改性	(3)
可测试性	(4)
安全性	c
易用性	(5)

表 1-1 质量属性识别表

【问题 2】 (7 分)

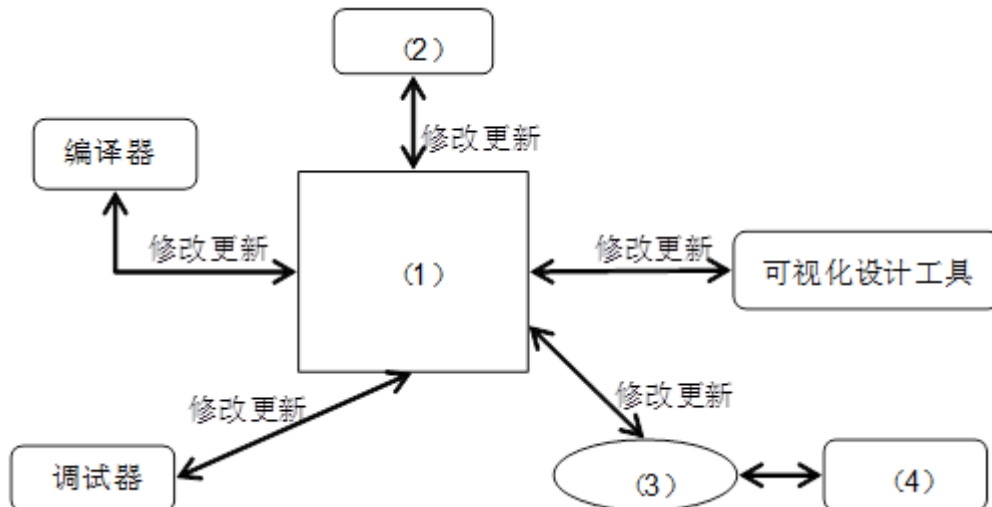
请在阅读题干需求描述的基础上，从交互方式、数据结构、控制结构和扩展方法 4 个方面对两种架构风格进行比较，填写表 1-2 中 (1) ~ (4) 处的空白。

比较因素	管道—过滤器风格	数据仓储风格
交互方式	顺序结构或有限的循环结构	(1)
数据结构	(2)	文件或模型
控制结构	(3)	业务功能驱动
扩展方法	接口适配	(4)

表 1-2 两种架构的比较

【问题 3】 (8 分)

在确定采用数据仓库架构风格后，王工给出了集成开发环境的架构图。请填写图 1-1 中 (1) ~ (4) 处的空白，完成该集成开发环境的架构图。



1-1 集成开发环境架构图

参考答案与试题解析

参考答案：

【问题 1】

(1) f (2) 性能 (3) d (4) g (5) b

【问题 2】

(1) 星型（工具之间通过中心结点进行交互）

(2) 数据流（或流式数据）

(3) 数据驱动

(4) 模型适配（与数据仓储进行数据适配）

【问题 3】

(1) 模型/数据库/语法结构树

(2) 编辑器

(3) 适配器

(4) 应用模拟器工具

试题解析：

本题考查的是架构设计过程中涉及到的一些质量属性，以及架构风格的对比。

常用的质量属性包括：

1、性能

性能 (performance) 是指系统的响应能力，即要经过多长时间才能对某个事件做出响应，或者在某段时间内系统所能处理的事件的个数。

2、可靠性

可靠性 (reliability) 是软件系统应用或系统错误面前，在意外或错误使用的情况下维持软件系统的功能特性的基本能力。

3、可用性

可用性 (availability) 是系统能够正常运行的时间比例。经常用两次故障之间的时间长度或在出现故障时系统能够恢复正常的速度来表示。

4、安全性

安全性 (security) 是指系统在向合法用户提供服务的同时能够阻止非授权用户使用的企图或拒绝服务的能力。安全性又可划分为机密性、完整性、不可否认性及可控性等特性。

5、可修改性

可修改性 (modifiability) 是指能够快速地对系统性能价格比进行变更的能力。通常以某些具体的变更为准，通过考察这些变更的代价衡量可修改性。

6、易用性

软件开发工具应有十分友好的用户界面,用户乐于使用;工具应能剪裁和定制,以适应特定用户的需要;工具应能提示用户的交互操作,提供简单有效的执行方式;工具还应能检查用户的操作错误,尽可能自动改正错误。

题目中还涉及到管道过滤器风格与仓库风格的对比。这两种风格的基本概念为:

在管道/过滤器风格的软件体系结构中,每个构件都有一组输入和输出,数据输入构件,经过内部处理,然后产生数据输出。因此,这里的构件被称为过滤器,这种风格的连接件就像是数据流传输的管道,将一个过滤器的输出传到另一过滤器的输入。

在仓库(repository)风格中,有两种不同的构件:中央数据结构说明当前状态,独立构件在中央数据存储上执行。

一方面,若构件控制共享数据,则仓库是一传统型数据库;另一方面,若中央数据结构的当前状态触发进程执行的选择,则仓库是一黑板系统。

题目中对比的方面分别是:

交互方式:管理过滤器很明显是顺序结构或循环结构,数据在管理中进行传递。而仓库结构是数据在中心位置,所有的处理均是中心结点与周边结点之间的交互,从形态来看,是星型的。

数据结构:从数据结构来看,仓库风格会使用一个文件将数据保存起来,所有的操作围绕这个文件进行。而管道过滤器则是在过滤器之间传递数据流。

控制结构:从控制结构来说仓库风格是业务功能驱动,而管道过滤器是由数据流驱动的。

扩展方法:从扩展方法来讲,管道过滤器是通过过滤器提供标准接口与其他过滤器对接,而数据仓库风格,要共享数据,扩展功能,只要功能的操作与数据模型本身是匹配的就行了,就像我们要共享一个数据库做系统集成,此时共享同一数据库的多个应用系统所用的数据模型一定会是一致的,否则无法去共享。

本题最后一问是分析开发环境架构图。要填充这个图,首先得想明白的是,这个环境中,仓库的中心是数据,数据对应的是语法结构树,因为一系列的处理都是以此为依据来进行的。接下来找出与编译器同等地位的其他处理工具即可。